Stripe Idempotent API Architecture

Eliminating Double Charges in Distributed Payment Infrastructure

f(f(x)) = f(x)	0 Double Charges
MATHEMATICAL IDEMPOTENCY GUARANTEE	GUARANTEED SAFE CLIENT RETRIES OVER DROPPED TCP

1. The Two Generals' Payment Retry Dilemma

In distributed payment processing, network unreliability is an unavoidable physical reality. When a client issues an HTTP `POST /v1/charges` request, the server executes the funds transfer against banking rails. If the network drops immediately prior to returning the HTTP 200 OK response, the client experiences a connection timeout.

- **If the client retries naively:** The customer risks being billed twice.
- **If the client fails to retry:** The transaction is lost, causing conversion abandonment.

2. HTTP Idempotency Matrix

HTTP Verb	Idempotent?	Safe (Read-Only)?	Distributed Behavior
`GET`	Yes	Yes	Pure read; multiple requests cause zero mutations.
`PUT`	Yes	No	Full entity overwrite; state identical on repetition.
`DELETE`	Yes	No	Deleting an ID repeatedly leaves the resource deleted.
`POST`	**No**	**No**	**Inherently non-idempotent: appends financial records.**

3. The Idempotency-Key Protocol

Stripe popularized the client-generated `Idempotency-Key: <UUID>` header. An idempotency interceptor inspects this key prior to business logic, converting non-idempotent `POST` requests into safe, repeatable operations.

4. The Idempotency Finite State Machine

The core idempotency engine transitions records through atomic database states:

Three Execution Branches

Branch 1 (New Key): Atomic `INSERT ... status = 'PROCESSING'`. Execute charge downstream. On completion, update `status = 'COMPLETED'` with serialized response body.

Branch 2 (Completed Key): Verify payload hash. If matching, **short-circuit execution** and replay cached response body with header `Idempotent-Replay: true`.

Branch 3 (In-Progress Key): Concurrent retry detected. Block on row lock or return `HTTP 409 Conflict`.

5. SHA-256 Payload Hash Verification

To prevent accidental key reuse or parameter drift (e.g., buggy client reusing the same key for $100 instead of $10), Stripe computes a canonical SHA-256 digest:

```
H_req = SHA256( HTTP_Method || Path || Canonical_Sorted_JSON(Body) )
```

If an existing completed key is matched but $H_{\text{new}} \ne H_{\text{stored}}$, execution is halted and the server returns `HTTP 400 Bad Request: Idempotency key reused with different parameters`.

6. Handling Zombie Locks and Host Crashes

If an API worker crashes after acquiring the `PROCESSING` lock but before marking it `COMPLETED`, the lock records a timestamp. If a lock exceeds a lease duration (e.g., 90 seconds), recovery workers query banking rails with a deterministic reference before re-opening the lock.

7. Production Implementation: PostgreSQL Middleware

```
-- Core Idempotency Storage Schema
CREATE TABLE idempotency_records (
    account_id VARCHAR(64) NOT NULL,
    idempotency_key VARCHAR(128) NOT NULL,
    request_hash CHAR(64) NOT NULL,
    status VARCHAR(16) NOT NULL, -- 'PROCESSING', 'COMPLETED'
    response_code INT,
    response_body JSONB,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    locked_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (account_id, idempotency_key)
);
```

8. Execution Latency: First Attempt vs. Replay

Execution Path	Banking Rails Touched?	Observed p99 Latency	Side Effect Count
Initial Request (First Arrival)	Yes (Card network authorization)	420 ms	Exactly 1 charge
Client Retry (Short-Circuit Replay)	**No (Bypassed entirely)**	**1.8 ms**	**0 duplicate charges**
Mismatched Parameter Re-attempt	No (Rejected at gateway)	0.9 ms	0 duplicate charges

9. Production Architectural Guardrails

1. Enforce Key Expiration (The 24-Hour TTL)
Idempotency keys should expire after 24 hours. Historical financial audits are preserved in immutable ledger tables, but the hot idempotency key table remains compact and RAM-resident.

2. Scope Keys Strictly to Authenticated Tenants
Always enforce composite primary keys: `(account_id, idempotency_key)`. This prevents cross-tenant key guessing or collision attacks between separate merchants.